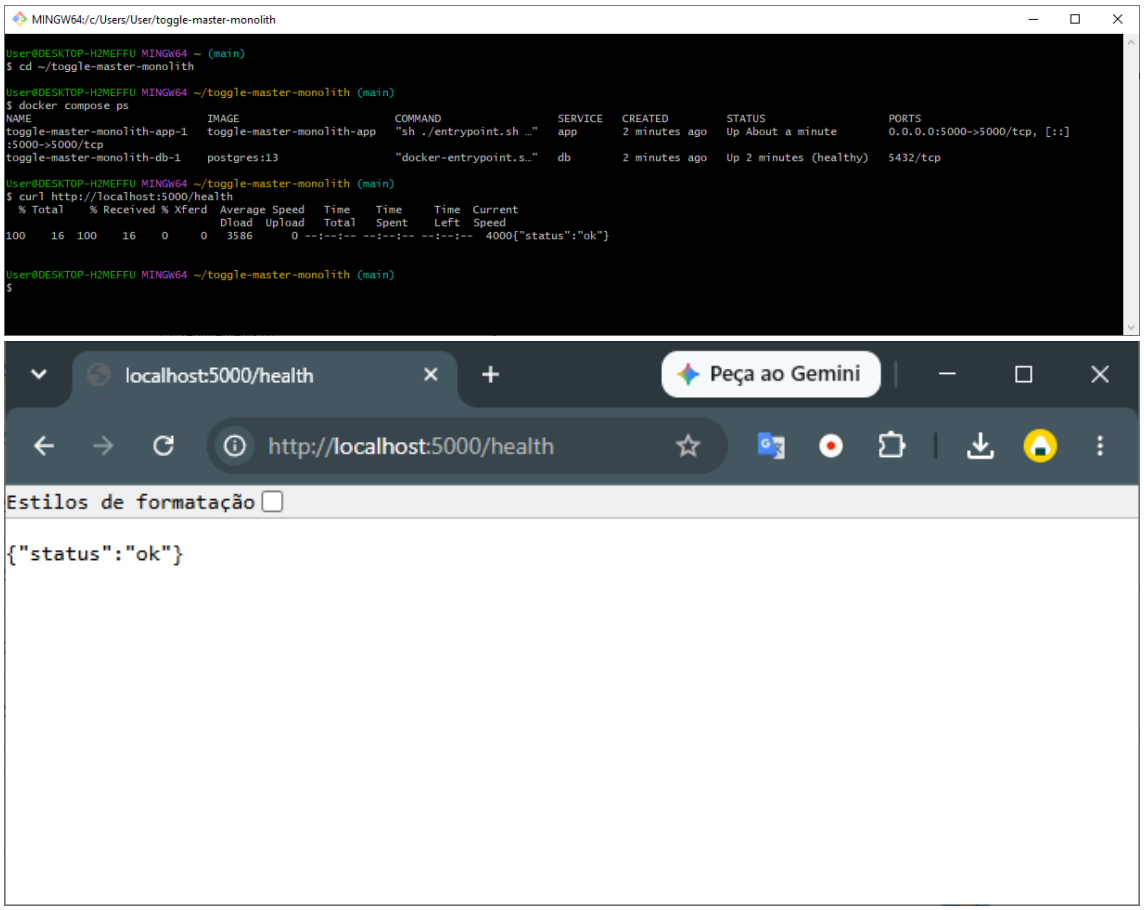
Cultura DevOps e Arquitetura de Aplicações:

Execução local da aplicação

A aplicação ToggleMaster foi executada localmente utilizando Docker Compose, responsável por subir os serviços da aplicação e do banco de dados PostgreSQL. Após a inicialização dos containers, foi validado o funcionamento da API por meio do endpoint /health, acessado em http://localhost:5000/health, que retornou o status ok, confirmando que a aplicação estava disponível e funcionando corretamente.

Durante a execução local, foi necessário realizar um ajuste no arquivo entrypoint.sh devido à diferença de quebra de linha entre Windows e Linux. Após a correção, os containers da aplicação e do banco de dados iniciaram normalmente.



Análise da arquitetura monolítica

A aplicação ToggleMaster é considerada monolítica porque concentra, em uma única aplicação, os endpoints da API, as regras de negócio relacionadas às feature flags e a comunicação com o banco de dados. Dessa forma, suas funcionalidades são executadas e implantadas como uma única unidade.

O arquivo app.py centraliza a implementação da API e o acesso aos dados, enquanto o PostgreSQL é utilizado como serviço de persistência. Embora o banco de dados seja executado em um container separado no ambiente local, a aplicação em si permanece monolítica, pois suas funcionalidades não estão divididas em serviços independentes.

Para um MVP, essa abordagem apresenta vantagens importantes, como menor complexidade inicial, facilidade de desenvolvimento, simplicidade no processo de deploy e menor esforço operacional. Essas características permitem disponibilizar a solução rapidamente para validação.

Por outro lado, à medida que a aplicação cresce, a arquitetura monolítica pode apresentar limitações, como maior acoplamento entre funcionalidades, dificuldade de escalar partes específicas da aplicação, necessidade de realizar o deploy da aplicação inteira mesmo para pequenas alterações e maior impacto potencial de falhas.

Nesse cenário, a escolha de uma arquitetura monolítica é adequada para a primeira versão do ToggleMaster, pois o objetivo desta fase é validar rapidamente o funcionamento da plataforma antes de evoluir para uma arquitetura mais distribuída.

Isso está alinhado ao próprio cenário do desafio: nesta primeira fase, o ToggleMaster é propositalmente um **MVP monolítico**, que depois será evoluído ao longo do curso.

Depois dessa seção, eu seguiria imediatamente para a próxima exigência: **12-Factor App**. Essa é a parte que vale a pena fazermos com mais cuidado, fator por fator, olhando o código real do projeto.

Análise dos 12-Factor App

A metodologia 12-Factor App define boas práticas para o desenvolvimento de aplicações modernas, portáveis e adequadas para ambientes de nuvem. A aplicação ToggleMaster foi analisada considerando esses princípios.

Fator	**Situação**	**Análise**
I. Codebase	Atende	O código da aplicação é versionado em um único repositório Git.
II. Dependencies	Atende	As dependências Python são declaradas no arquivo requirements.txt.

Fator	Situação	Análise
III. Config	Atende parcialmente	As configurações do banco são fornecidas por variáveis de ambiente, evitando credenciais diretamente no código.
IV. Backing Services	Atende	O PostgreSQL é tratado como um serviço externo à aplicação.
V. Build, Release, Run	Atende parcialmente	Existe separação básica entre construção da imagem e execução, porém não há pipeline automatizado de CI/CD nesta fase.
VI. Processes	Atende parcialmente	A aplicação é executada como um processo por meio do Gunicorn, mas ainda depende de uma implantação simples.
VII. Port Binding	Atende	A API é disponibilizada diretamente pela porta 5000.
VIII. Concurrency	Atende parcialmente	O Gunicorn permite execução por processos, porém não há estratégia definida de escalabilidade horizontal.
IX. Disposability	Atende parcialmente	Os containers podem ser iniciados e encerrados rapidamente, mas não há configuração avançada de graceful shutdown ou resiliência.
X. Dev/Prod Parity	Atende parcialmente	O ambiente local utiliza Docker Compose, enquanto a implantação proposta na AWS será realizada diretamente em uma EC2 com RDS.
XI. Logs	Atende parcialmente	Os logs são enviados para a saída padrão, porém não existe nesta fase uma solução centralizada de observabilidade.
XII. Admin Processes	Atende parcialmente	Não há definição estruturada de processos administrativos ou tarefas pontuais executadas separadamente da aplicação.

Conclusão: A aplicação já incorpora parte relevante dos princípios do 12-Factor App, especialmente no versionamento do código, declaração de dependências, uso de variáveis de ambiente e tratamento do banco como serviço externo. Por se tratar de um MVP, alguns fatores ainda são atendidos apenas parcialmente e poderão evoluir

nas próximas fases, principalmente em automação, escalabilidade, observabilidade e paridade entre ambientes.

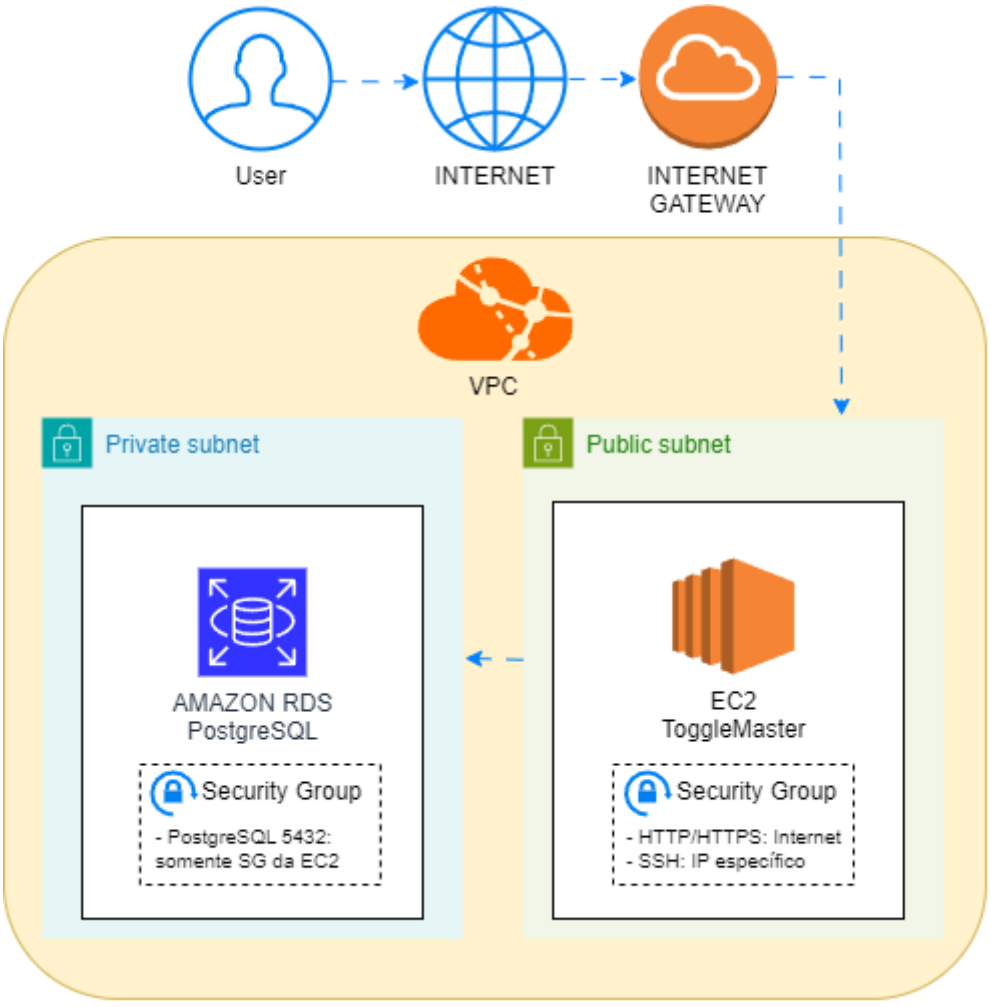
Arquitetura proposta na AWS

A arquitetura do ToggleMaster foi projetada utilizando serviços da AWS com separação entre os componentes expostos à Internet e os componentes internos da aplicação. A solução será provisionada dentro de uma **Amazon VPC**, contendo uma sub-rede pública e uma sub-rede privada.

A aplicação ToggleMaster será executada em uma instância **Amazon EC2**, localizada na subnet pública. Essa instância será responsável por hospedar a API da aplicação e receber as requisições dos usuários através da Internet.

O acesso externo à VPC será realizado por meio de um **Internet Gateway**, permitindo que os usuários alcancem a instância EC2.

O banco de dados será hospedado utilizando **Amazon RDS for PostgreSQL**, localizado na subnet privada. Dessa forma, o banco não ficará diretamente exposto à Internet, sendo acessado somente pela aplicação executada na EC2.



Fluxo de comunicação

O fluxo principal da aplicação ocorre da seguinte forma:

Usuário

↓

Internet

↓

Internet Gateway

↓

EC2 - ToggleMaster

↓

PostgreSQL TCP 5432

↓

Amazon RDS PostgreSQL

A instância EC2 recebe as requisições da aplicação e realiza a comunicação com o banco de dados PostgreSQL hospedado no RDS.

Segurança da arquitetura

Foram definidos Security Groups distintos para a aplicação e para o banco de dados, restringindo a comunicação de acordo com a necessidade de cada componente.

Security Group da EC2

O Security Group associado à instância EC2 permitirá:

HTTP/HTTPS → acesso pela Internet

SSH → somente de um endereço IP específico

O acesso SSH será restrito para reduzir a exposição administrativa da instância.

Security Group do RDS

O Security Group associado ao Amazon RDS permitirá somente:

PostgreSQL

TCP 5432

Origem: Security Group da EC2

Dessa forma, o banco de dados não aceitará conexões diretamente da Internet. Somente recursos associados ao Security Group da EC2 poderão iniciar conexões com o PostgreSQL.

Essa configuração atende diretamente à exigência do desafio de permitir no RDS apenas tráfego proveniente do Security Group da EC2.

Justificativa das sub-redes

A instância EC2 foi posicionada na **subnet pública** porque precisa receber requisições externas para disponibilizar a API do ToggleMaster.

O Amazon RDS foi posicionado na **subnet privada**, pois o banco de dados não necessita ser acessado diretamente pelos usuários. O acesso ocorre exclusivamente por meio da aplicação hospedada na EC2.

Essa separação reduz a superfície de exposição da infraestrutura e aplica um princípio básico de segurança em nuvem: disponibilizar publicamente apenas os componentes que realmente necessitam de acesso externo.

Resumo da arquitetura

Componente	**Função**	**Localização**
VPC	Isolamento lógico da infraestrutura	AWS
Internet Gateway	Comunicação entre Internet e VPC	VPC
EC2	Hospedagem da aplicação ToggleMaster	Public Subnet
RDS PostgreSQL	Persistência dos dados	Private Subnet
SG EC2	Controle de acesso à aplicação	EC2
SG RDS	Controle de acesso ao banco	RDS